

IIO 触发缓冲与 DRDY 硬件中断详解

从"为什么需要"到上板实战 —— 以 MPU-6050 + STM32MP157 为例

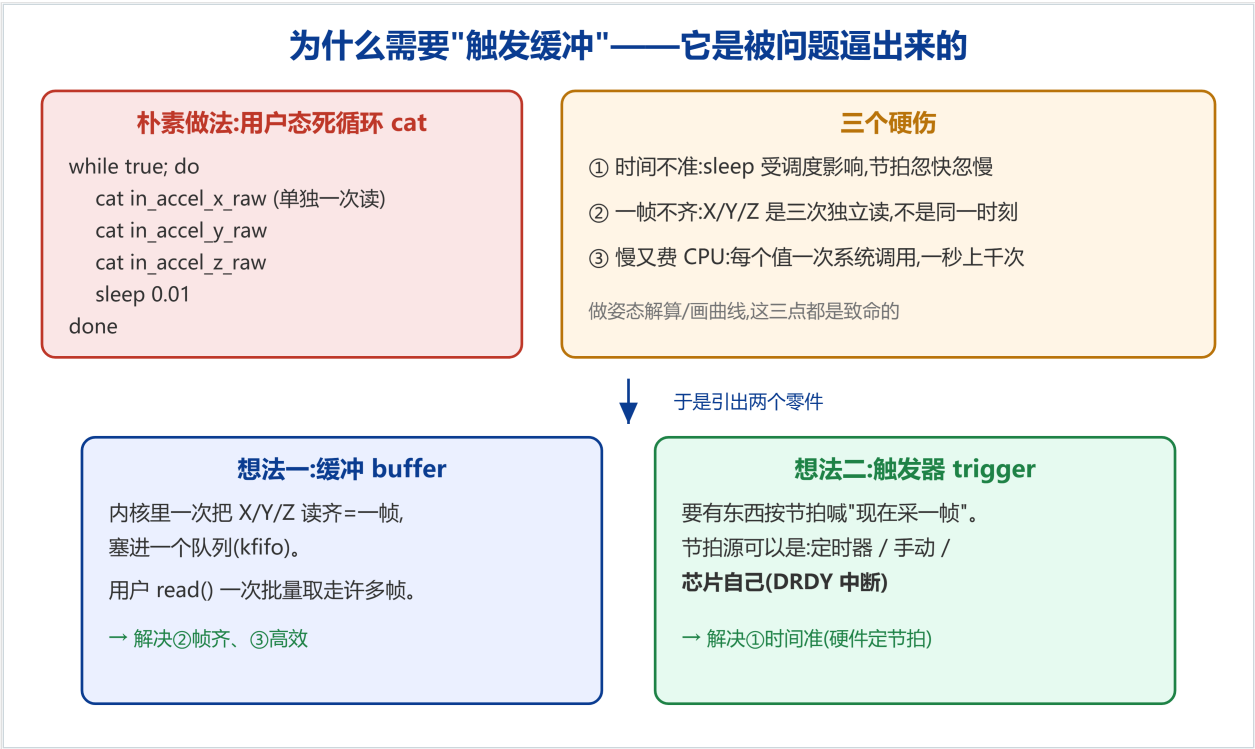
MPU-6050 IIO 驱动学习 · 进阶篇 · 2026 年 10 月

导读

本文把"单次读 (cat in_accel_x_raw)"之外的**流式采集能力**讲透:它依赖两个核心机制——**触发器(trigger)**与**触发缓冲(triggered buffer)**,数据由芯片的 **DRDY(数据就绪)硬件中断**驱动,经 /dev/iio:deviceN 成帧输出。全文按"为什么需要 → 三个概念 → 零件归属 → 芯片侧vs板子侧 → 硬件接线 → 软件三件事 → 运行时数据流 → 上板验证 → 面试要点"的逻辑展开。

一、为什么需要触发缓冲(它是被问题逼出来的)

需求很朴素:**连续不断地**拿到传感器数据(如每 10ms 一帧)去画曲线、做姿态解算。最先想到的是用户态死循环 cat ,但它有三个硬伤,正是这三个问题逼出了缓冲与触发器两个零件。



2.2 触发器 trigger —— 喊"现在采一帧"的节拍器

一个决定**何时**采的节拍源。节拍可来自:内核定时器、人工 sysfs、或**芯片自己(DRDY 中断)**。本项目用 DRDY:芯片真正测完一帧才拉 INT 脚通知,采集节拍与芯片采样完美同步,时间最准。它在系统里是独立设备 `/sys/bus/iio/devices/trigger0`,名字 `mpu6050-dev0`。

2.3 pollfunc —— 连接触发器与缓冲的"工单"

`struct iio_poll_func` 是一张工单:**节拍一到,照着它干活**。它本身不产生节拍、也不是队列,是二者之间的胶水,记了四样东西:

```
struct iio_poll_func {
    irqreturn_t (*h)(...);           // 顶半部 = iio_pollfunc_store_time(记时间戳)
    irqreturn_t (*thread)(...);       // 底半部 = mpu6050_trigger_handler(读数据→塞缓冲)
    struct iio_dev *indio_dev;        // 回指:这单子属于哪个设备
    s64 timestamp;                   // 顶半部记下"这一刻",底半部拿去用
};
```

解耦思想:触发器只负责喊"干!",具体干什么全写在 pollfunc 工单里。于是**节拍源可随意更换**(DRDY / 定时器 / 手动),采集动作的代码一行不改——这正是把三者拆开的价值。

三、三个零件分别放在哪里

关键区别:buffer 和 pollfunc 属于"你的设备"(挂在 `indio_dev` 下);trigger 是独立设备。

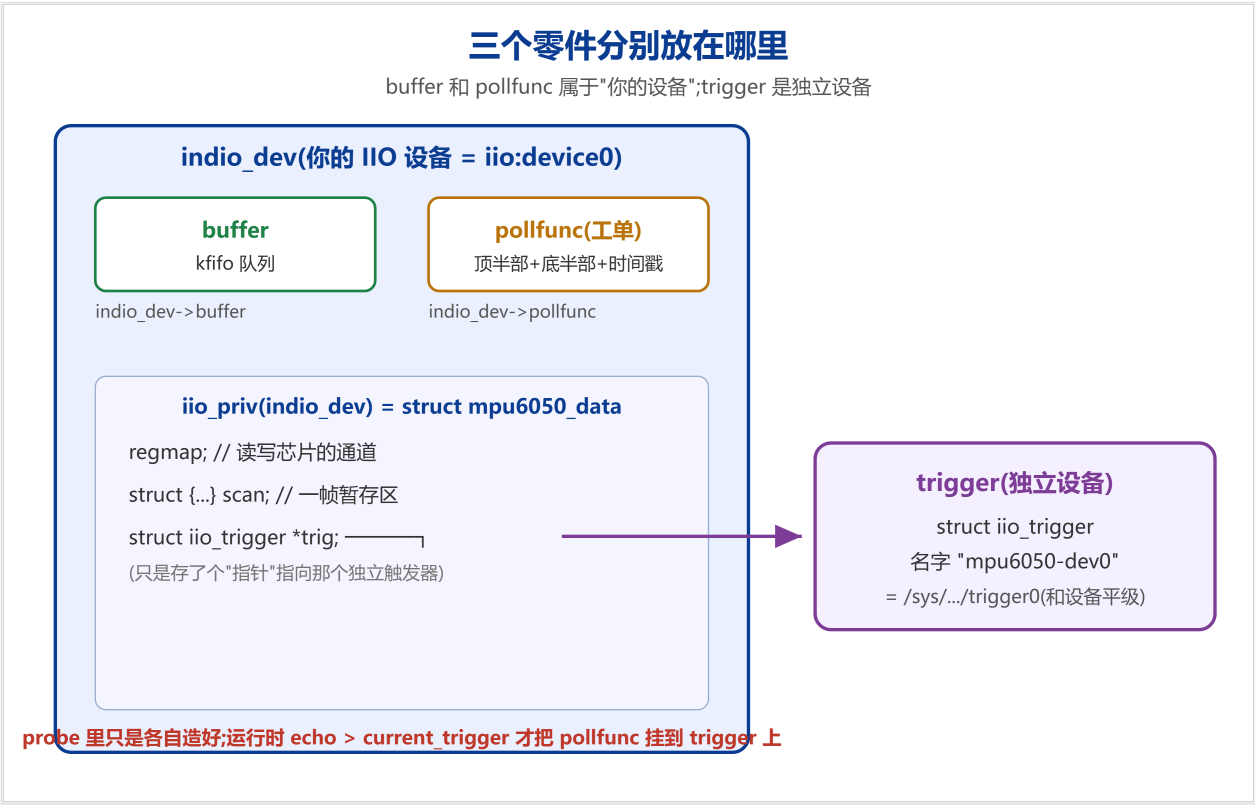


图 2 零件归属:buffer/pollfunc 在 `indio_dev` 内,trigger 独立存在

零件	C 结构	指针存在哪	谁创建	系统里露头处
缓冲	<code>struct iio_buffer</code>	<code>indio_dev->buffer</code>	<code>devm_iio_triggered_buffer_setup</code>	<code>iio:device0/buffer/</code> 、 <code>/dev/iio:device0</code>
pollfunc	<code>struct iio_poll_func</code>	<code>indio_dev->pollfunc</code>	同上(一起造)	无独立 sysfs(内部胶水)
触发器	<code>struct iio_trigger</code>	<code>data->trig</code> + 注册进触发器子系统	<code>devm_iio_trigger_alloc</code>	独立的 <code>trigger0</code>

三者 在 `probe` 时只是各自造好;真正把 `pollfunc` "插"到触发器上是 **运行时**:`echo mpu6050-dev0 > .../trigger/current_trigger` 这一刻,IIO 才把 `indio_dev->pollfunc` 挂进触发器的消费者列表。这也是 `ls /sys/bus/iio/devices/` 会看到 `iio:device0` 和 `trigger0` 两个平级条目的原因。

四、芯片侧 vs 开发板侧(最容易混的地方)

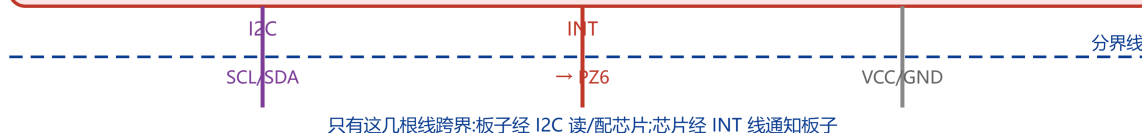
一句话结论:buffer / trigger / pollfunc 全部在开发板(STM32MP157 主核)这边,是内核 RAM 里的软件对象;MPU6050 芯片里一个都没有。芯片里只有:配置寄存器 + 数据寄存器 + 内部采样 + 一根 INT 引脚。

芯片侧 vs 开发板侧 —— 谁在哪里

MPU6050 芯片(GY-521 小蓝板)—— 纯硬件,没有软件

- 内部 ADC:按设定频率(100Hz)不停测量
- 配置寄存器:SMPLRT_DIV / CONFIG / INT_PIN_CFG / INT_ENABLE
- 数据寄存器:ACCEL / TEMP / GYRO(0x3B~0x48)
- INT 引脚(输出):测完一帧就按配置拉高一下

芯片里**没有** buffer / trigger / pollfunc —— 它根本不知道这些概念



STM32MP157 开发板 / 主核 A7 / 跑 Linux

硬件外设:

I2C4 控制器(走 SCL/SDA 读写芯片寄存器) · GPIO/EXTI:PZ6 收 INT 上升沿 → 产生内核中断

纯软件(你的 .ko + 内核,全在板子 RAM 里):

驱动 mpu6050_iio.ko · regmap · iio_dev

★ buffer(队列) · trigger(trigger0) · pollfunc(工单)

中断处理函数 · trigger_handler · /dev/iio:device0 · /sys/bus/iio/...

判断口诀:凡"对象/队列/函数/概念"都在板子软件里;芯片里只有"寄存器+采样+INT一根脚"

图 3 芯片侧 vs 板子侧 —— 谁在哪里,两边只靠 I2C 与 INT 线相连

一个易绕的点:probe 里 `regmap_write(..., SMPLRT_DIV, 0x09)` 这类语句——**执行代码的是板子 CPU(软件),但它经 I2C 伸手去改芯片内部的寄存器,结果落在芯片里。**即"配置动作在板子发起,配置结果存在芯片"。这正是"驱动是桥"的含义。

跨越分界线的只有两处:**芯片→板子**是 INT 线(芯片喊"我好了");**板子→芯片**是 I2C 读/写(板子去读数据、去配置)。其余所有概念都在板子软件里,芯片对它们一无所知。

五、硬件准备:把 INT 接到板子 GPIO

GY-521 模块的 INT 脚(数据就绪输出)此前悬空,需用一根线接到板子一个空闲 GPIO。本板(正点原子 ATK-DLMP157 V1.5)扩展排针 JP1 上:

- JP1 第 25 脚 = I2C4_SCL(PZ4)、第 26 脚 = I2C4_SDA(PZ5) —— SCL/SDA 已接。
- 第 28 脚 = PZ6 —— 无信号名的空闲 GPIO,选它接 INT。(第 5 脚 6D_INT/PA14 被板载 ICM-20608D 占用,不能用。)

接线:模块 INT —> JP1 第 28 脚(PZ6)。MPU6050 的 INT 默认推挽、高有效、50μs 脉冲,自带驱动能力,不需外接上/下拉电阻;且是"模块输出→SoC 输入",接错也不会烧,最坏是收不到中断。

六、软件实现:三件事

6.1 设备树:告诉内核"中断在 PZ6"

在 mpu6050@68 节点加两行(PZ6 属 GPIOZ,运行时已确认该 bank 使能,line 6):

```
mpu6050@68 {
    compatible = "alientek,mpu6050";
    reg = <0x68>;
    interrupt-parent = <&gpioz>;          /* 中断挂在 GPIOZ 控制器上 */
    interrupts = <6 IRQ_TYPE_EDGE_RISING>; /* 第6号线=PZ6;上升沿触发 */
};
```

<6 ...> 的 6 是**组内线号**(PZ6 → 6),不是全局 gpio 号 406。编好 dtb 拷到 TFTP 目录,重启生效。之后 i2c 核心会自动把该中断号填进驱动的 client->irq。

6.2 配置芯片四个寄存器(让 DRDY 以稳定节拍产生)

寄存器(值)	作用	为什么这么设
CONFIG=0x03 (0x1A)	DLPF 数字低通滤波	DLPF_CFG=3:带宽约 44Hz 滤噪,且把基准频率定为 1kHz
SMPLRT_DIV=0x09 (0x19)	采样率分频	采样率 = 1000/(1+9) = 100Hz (每 10ms 一帧,拉一次 INT)
INT_PIN_CFG=0x00 (0x37)	INT 引脚电气特性	全 0 = 高有效 + 推挽 + 50μs 脉冲 → 对应设备树的 上升沿 、免上拉、免清中断
INT_ENABLE=0x00 (0x38)	中断使能开关	先全关;等用户开缓冲时由 set_trigger_state 把 bit0(DATA_RDY)置 1,实现"按需开中断"

CONFIG 与 SMPLRT_DIV 是一对:前者把基准定成 1kHz,后者除以 10 得到 100Hz 出数节拍。INT_PIN_CFG 的"高有效"必须与设备树的"上升沿"一致,否则收不到中断。

6.3 驱动三块代码与逻辑链

probe 里造三个零件,靠触发器这个"中枢"连起来:

```
/* ① 建触发缓冲:顶半部记时间戳,底半部=handler 读数据 */
devm_iio_triggered_buffer_setup(&client->dev, indio_dev,
    iio_pollfunc_store_time, mpu6050_trigger_handler, NULL);

/* ② 建 DRDY 触发器,并把 indio_dev 藏进去备用 */
data->trig = devm_iio_trigger_alloc(&client->dev, "%s-dev%d",
    indio_dev->name, indio_dev->id);
data->trig->ops = &mpu6050_trigger_ops; /* 带开关 set_trigger_state */
iio_trigger_set_drvdata(data->trig, indio_dev);

/* ③ 申请中断:INT上升沿 → generic_data_rdy_poll → 踢触发器 */
devm_request_irq(&client->dev, client->irq,
    iio_trigger_generic_data_rdy_poll, /* 中断来了只做一件事:踢 trig */
    IRQF_TRIGGER_RISING, "mpu6050_drdy",
    data->trig); /* ★把触发器当私有数据传入 */
```

连接关系:触发器是中枢,两头被接上——③把物理中断接成它的"上游节拍源"(靠传入 data->trig),①的 pollfunc 在运行时接成它的"下游消费者"。set_drvdata 则让开关函数 set_trigger_state 能从 trig 找回 indio_dev → 再到 regmap 去写芯片 INT_ENABLE。

七、运行时数据流(每步在哪一侧)

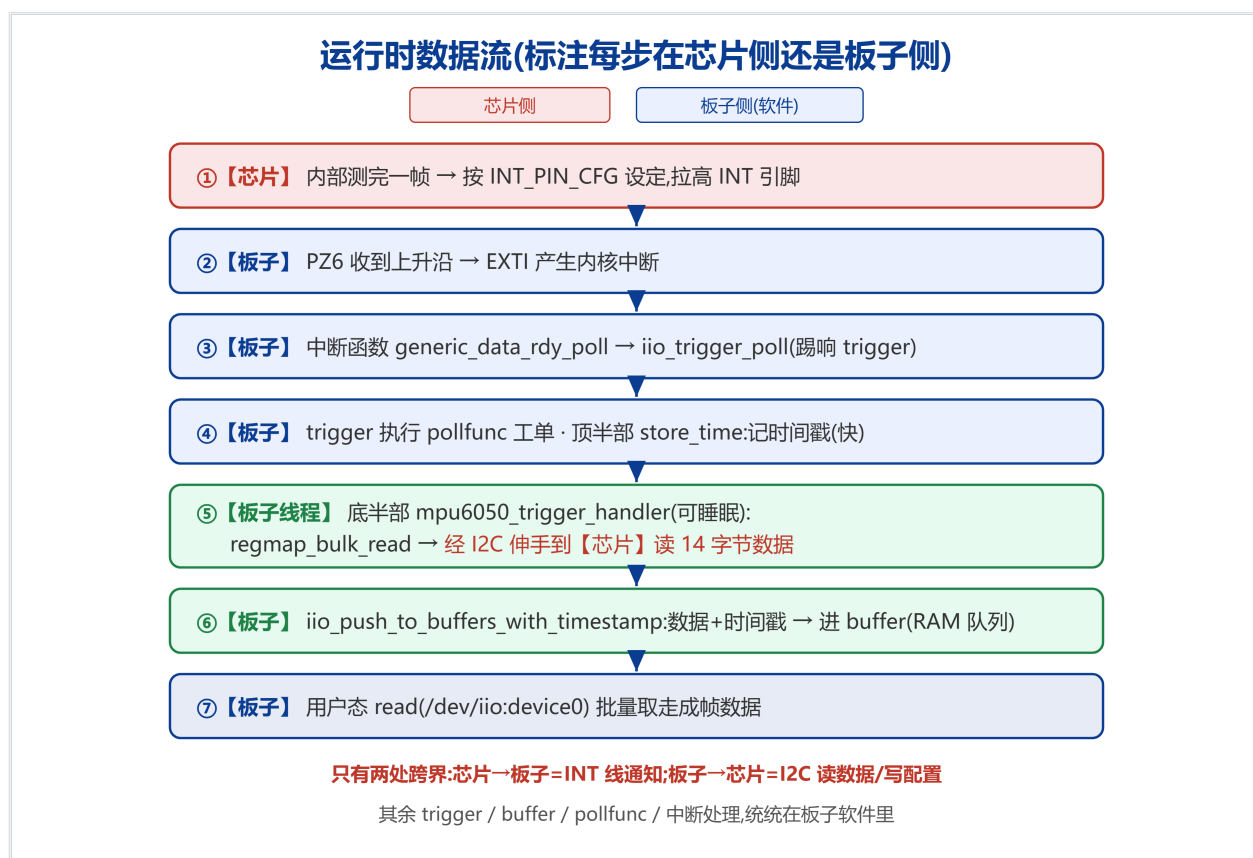


图 4 从芯片测完一帧到用户取走的完整时序,标注芯片侧/板子侧

底半部 mpu6050_trigger_handler 的核心:一次连续读 14 字节(AX,AY,AZ,TEMP,GX,GY,GZ),连同时间戳推进缓冲:

```
regmap_bulk_read(data->regmap, MPU6050_REG_ACCEL_XOUT_H,
                 data->scan.chans, sizeof(data->scan.chans));
iio_push_to_buffers_with_timestamp(indio_dev, &data->scan, pf->timestamp);
iio_trigger_notify_done(indio_dev->trig);
```

其中 `scan` 暂存区的 `scan_index` 与寄存器连续读顺序一致(0=AX...6=GZ,7=时间戳),时间戳按 8 字节对齐放尾部;各通道 `scan_type` 为 16 位、有符号、大端(芯片高字节在前)。

八、上板验证

```
cd /sys/bus/iio/devices/iio:device0
echo mpu6050-dev0 > trigger/current_trigger      # 把触发器绑给缓冲
echo 1 > scan_elements/in_accel_x_en             # 选通道:accel x/y/z + 时间戳
echo 1 > scan_elements/in_accel_y_en
echo 1 > scan_elements/in_accel_z_en
echo 1 > scan_elements/in_timestamp_en
echo 64 > buffer/length
echo 1 > buffer/enable                           # 开缓冲(此刻才打开芯片DRDY中断)
cat /proc/interrupts | grep mpu6050_drpy         # 关键证据:计数应 ~100/秒 在涨
timeout 1 hexdump -C /dev/iio:device0 | head     # 成帧二进制数据
echo 0 > buffer/enable
```

判断成功:① `mpu6050_drpy` 中断计数 1 秒涨约 100 → 证明是**硬件中断**在驱动采集,不是软件轮询;② `hexdump` 源源不断出数据 → `/dev/iio:device0` 成帧输出打通。每帧 16 字节 = AX/AY/AZ(各2字节有符号大端)+ 2 字节填充 + 8 字节时间戳(ns)。

九、面试要点

- **为什么要触发缓冲:**解决单次 `cat` 的时间抖动、帧不齐、系统调用开销三大问题。
- **trigger/buffer/pollfunc 分工:**触发器定节拍(可插拔)、缓冲排队、`pollfunc` 是二者胶水;解耦让节拍源可换而采集代码不变。
- **中断上下半部:**硬中断只 `iio_trigger_poll` 踢一下(极短);真正读 I2C(会睡眠)放底半部线程。
- **软硬件边界:**芯片只有寄存器+采样+INT 脚;trigger/buffer/pollfunc 都是板子内核软件;两处跨界=I2C 与 INT 线。
- **按需开中断:**probe 先关 `INT_ENABLE`,用户开缓冲时 `set_trigger_state` 才打开,省电不扰系统。

本文为 MPU-6050 IIO 驱动学习的进阶篇(触发缓冲 + 硬件中断),基于真实上板实战整理。配套代码见仓库 `examples/learning-driver/` 与 `drivers/iio/imu/mpu6050/`;系列文档见 `docs/learning/`。仓库:github.com/fyy0619-cell/xiangmu